1. Objetivo

Implementação do algoritmo de Banker (evitação de deadlock) em C para sistemas Unix/Linux, baseado na 10ª edição do livro Operating System Concepts. O programa processa a matriz de alocação máxima a partir de customer.txt, executa comandos em lote via commands.txt com suporte dinâmico a número de recursos e clientes, produzindo a saída formatada em result.txt

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Readme	OK	Conferir no repositório do github
Relatório	OK	Conferir o arquivo do relatório final (vgjda2.pdf)
Makefile	OK	Executar make e make clean
Tratamentos de erro	OK	Executar ./banker sem argumentos, com valores negativos ou arquivos ausentes/malformados, validando a emissão de mensagens claras em stderr, saída com erro (EXIT_FAILURE) e ausência de vazamento de memória (atexit).
Leitura flexível de recursos via CLI	OK	Executar ./banker 10 5 7 (3 recursos) ou com outra quantidade de parâmetros (ex.: ./banker 10 5 7 4) e verificar o dimensionamento dinâmico das estruturas de dados
Leitura de clientes via customer.txt	OK	Preencher customer.txt com linhas separadas por vírgula (ex.: 7,5,3), executar o programa e verificar no comando * se a matriz MAXIMUM e o vetor inicial NEED refletem exatamente os valores lidos
Processamento de comandos via commands.txt	OK	Executar o binário com comandos RQ (requisição), RL (liberação) e * (estado), validando o teste de segurança (esta_seguro), rollback em estado inseguro e bloqueios por excesso de demanda/recursos (evidencias.log).
Geração exata da saída em result.txt	OK	Rodar ./banker 10 5 7 com os arquivos base do enunciado e comparar cat result.txt

Requisito do enunciado	Status	Como testar / evidência
		caractere por caractere com o gabarito esperado, incluindo formatação de colunas e mensagens exatas.

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./banker 10 5 7
cat result.txt
```

4. Arquitetura

Arquivos e responsabilidades:

1. **banker.c → Contém as estruturas de dados, o algoritmo de segurança (evitação de deadlock), o parsing de argumentos/arquivos e a gravação da saída result.txt.**
2. **Makefile→ Regras automatizadas de compilação -std=c11 e `make clean`**
3. **evidencias.log → Contém a evidência de todos os testes e compilações da implementação.**
4. **README.md → Documentação técnica de compilação, execução e tratamento de casos limite.**

Decisões técnicas:

1. **Validação prévia total → Evitar estado parcial inconsistente → Se customer.txt ou commands.txt possuírem erros, o arquivo result.txt antigo é removido e a execução é cancelada com status 1.**
2. **Vetores contíguos em lote → Evitar fragmentação e facilitar gerenciamento de memória → Matrizes são alocadas como vetores indexados por cliente * m + recurso**
3. **Limpeza automatizada via atexit(limpa_recursos)--> Prevenir vazamentos de memória (memory leaks) → Garantia de encerramento limpo em qualquer ponto de saída (morrer/exit).**

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Desenvolvimento Incremental com Testes Modulares	Implementação sequencial e isolada de cada função()	Não houve.

5.2 Diário de Tentativas

#						
1	S1	Fazer estrutura(Structs) da implementação	Sucesso		10/11/2026 – 12:24:19 - 12:25:16	evidencias.log (linhas 1 - 46)
2	S1	Implementar parsing e validação de customer.txt e commands.txt	Sucesso		10/11/2026 – 14:49:34 - 15:08:59	evidencias.log (linhas 47 - 176)
3	S1	Teste de leitura com a main temporária com entradas válidas (./banker 10 5 7).	Sucesso		10/11/2026 – 14:49:32 - 15:08:59	evidencias.log (linhas 47 - 176)
4	S1	Teste de tratamento de erro com número de recursos incompatíveis (./banker 10 5).	Falha(esperada)	Validação da CLI acusou divergência com as colunas de customer.txt.	10/11/2026 – 14:49:32 - 15:08:59	evidencias.log (linhas 47 - 176)
5	S1	Teste de tratamento de erro com cliente inexistente no comando (RQ 9), o arquivo commands.txt foi editado antes do evidencias.log	Falha(esperada)	Validação de limites recusou ID de cliente maior que o total cadastrado.	10/11/2026 – 14:49:32 - 15:08:59	evidencias.log (linhas 47 - 176)
6	S1	Commit no GitHub (git add, git commit, git push).	Sucesso		10/11/2026 – 14:49:32 - 15:08:59	evidencias.log (linhas 47 - 176)
7	S1	Testar requisição RQ 0 4 4 4 utilizando a main temporária dos testes anteriores	Falha	O arquivo result.txt não foi gerado, pois a main temporária apenas imprimia diagnósticos no terminal em vez de gravar a saída.	10/11/2026 – 15:32:27 - 15:44:30	evidencias.log(177 - 259)
8	S1	Recompilar o projeto completo com make clean && make e reexecutar ./banker 5 5 5.	Falha	A requisição válida de 4 4 4 foi recusada e a matriz NEED foi impressa como 0 0 0 em result.txt. Causa: omissão do memcpy entre maximo e necessidade na	10/11/2026 – 15:32:27 - 15:44:30	evidencias.log(177 - 259)

#						
				função inicializa_banco.		
9	S1	Compilar banker.c com o main temporário sem fazer as funções de impressão.	Falha	O compilador acusou implicit declaration of function para imprime_unido e imprime_espacado , além de avisos de funções não utilizadas (ler_clientes, ler_comandos).	11/11/2026 – 14:45:29 - 14:54:03	evidencias.log(260 - 404)
10	S1	Coloca funções e testar isoladamente com recursos iniciais em [3, 3, 2].	falha	O algoritmo indicou falha em estado inicial: INSEGURO e negou a requisição 1 0 2. Causa: a disponibilidade inicial (3, 3, 2) era inferior à demanda máxima do cliente 0 (7,5,3)	11/11/2026 – 14:45:29 - 14:54:03	evidencias.log(260 - 404)
11	S1	Ajustar o main temporário para simular recursos suficientes (10, 5, 7) e reexecutar.	sucesso		11/11/2026 – 14:45:29 - 14:54:03	evidencias.log(260 - 404)
12	S1	Integrar execução de comandos e formatação de saída e main no banker.c para gerar a saída final em result.txt.	Falha	O compilador recusou porque as funções imprime_unido, imprime_espacado e executa_requisicao ficaram duplicadas durante a junção dos trechos de código.	11/11/2026 – 16:45:28 - 17:01:04	evidencias.log(405 - 555)
13	S1	Unificar o código do banker.c, remover duplicidades e executar ./banker 10 5 7	sucesso		11/11/2026 – 16:45:28 - 17:01:04	evidencias.log(405 - 555)
14	S1	Teste de Caso Limite: requisição de 100% dos recursos (RQ 0 10 5 7) seguida de liberação total.	sucesso		11/11/2026 – 17:55:34 - 18:00:29	evidencias.log(556 - 659)

#						
15	S1	Teste de Caso Inválido: requisição 4 2 2 excedendo a demanda máxima 3 2 2 do cliente 0.	sucesso		11/11/2026 – 17:55:34 - 18:00:29	evidencias.log(556 - 659)

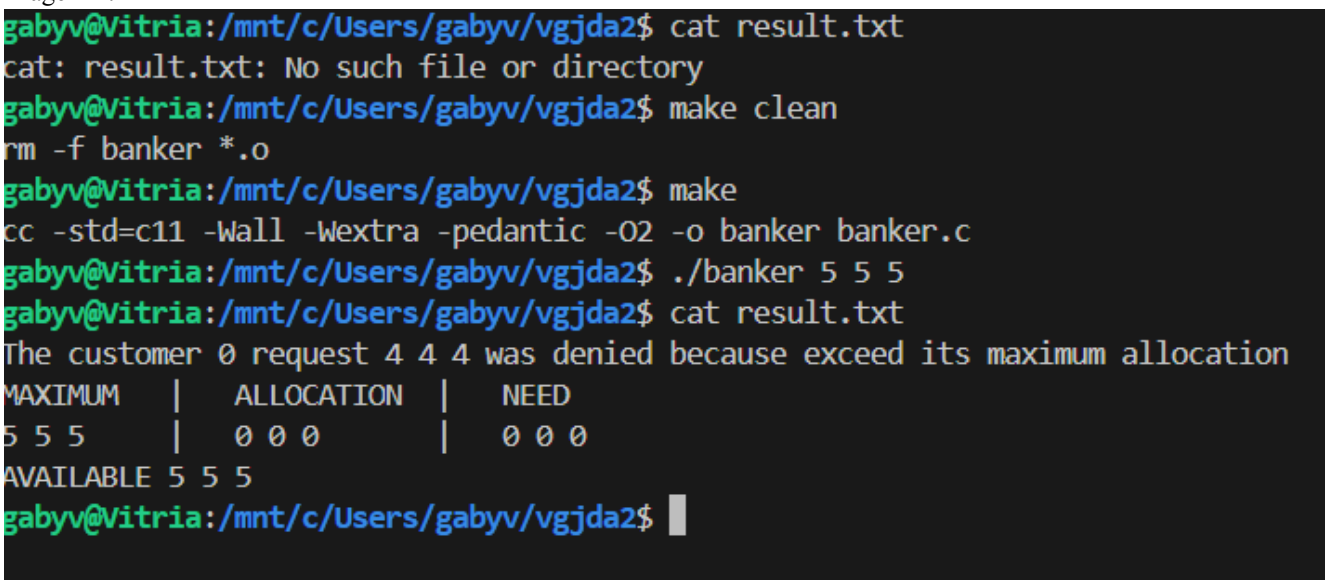
6. Evidências

6.1 Log automático

O registro de execuções de compilação, testes de casos válidos e inválidos foi capturado e armazenado no arquivo evidencias.log anexo ao pacote da entrega.

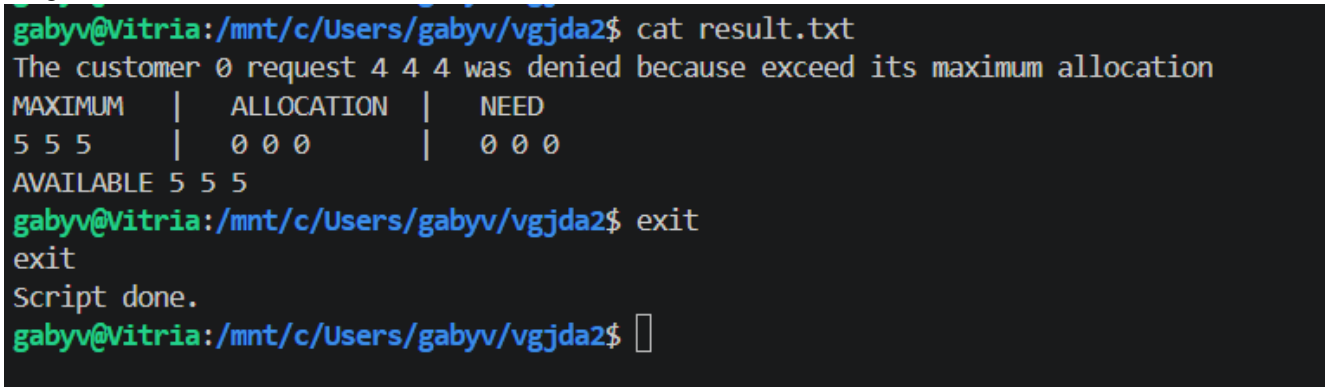
6.2 Prints

Imagem 1:



Legenda 1: Evidência de falha.. O pedido do cliente 0 (4 4 4) é incorretamente recusado alegando excesso da alocação máxima (exceed its maximum allocation).

Imagem 2:



Legenda 2: O programa compilou de forma limpa e gerou result.txt igual ao exemplo do enunciado, conferido visualmente.

7. Uso de IA

Onde usei IA:

Usei IA para entender os principais assuntos relacionados a essa implementação, também foi utilizado para ajudar a construir algumas partes do código, em main temporário para fazer testes e na estruturação do relatório e na escrita conforme os critérios de avaliação.

Prompts principais:

- **Me explique o conteúdo que está relacionado a essa implementação, o assunto é deadlock, mas diga/explique o assunto que eu tenho que estudar para fazer essa implementação.**
- **Me ajude nessa parte do enunciado, não estou conseguindo formular a estrutura do código**
- **Me diga como testar essa parte do código(em seguida ela me dava um main temporário)**
- **Porque deu esse resultado(mandava o print do resultado) e como mudar isso.**
- **Mande um teste de limite e inválido para eu fazer os testes.**

O que validei manualmente e como:

- **A compilação limpa do código com make e executei testes manuais de validação de leitura dos arquivos customer.txt e commands.txt. Testei cenários de sucesso (processando 5 clientes e 3 comandos) e forcei casos de erro de entrada via linha de comando, como passar 2 recursos ao invés de 3 (./banker 10 5) e tentar acessar um cliente inexistente (cliente 9).**

- **Inicialmente, verifiquei que a main de testes não produzia o arquivo de saída result.txt. Em seguida, integrei a estrutura completa do main temporário com make clean && make e executei o programa com a entrada de teste (customer.txt contendo 5,5,5 e commands.txt com RQ 0 4 4 4) .**

- **Inspecionei manualmente o arquivo result.txt gerado via cat result.txt e identifiquei a falha de lógica na inicialização do banco: o sistema recusou a requisição acusando excesso de alocação máxima (was denied because exceed its maximum allocation) devido à ausência da cópia inicial dos dados para a matriz de necessidade (NEED), que permaneceu zerada (0 0 0).**

- **Validação Isolada do Algoritmo de Segurança:utilizei um main de teste temporário diretamente. Inicialmente, a compilação falhou devido à declaração implícita de funções de formatação (imprime_unido e imprime_espacado). Após reordenar as funções e ajustar os recursos iniciais para [10, 5, 7], validei manualmente no terminal que a função esta_seguro() reconheceu a matriz NEED preenchida como [OK] SEGURO e autorizou a alocação Allocate to customer 0 the resources 1 0 2.**

- **Caso Limite: Criei o diretório teste_limite/ com um cliente exigindo 10 5 7. Validei via cat result.txt que o programa permitiu alocar todos os recursos do banco (AVAILABLE zerado em 0 0 0), mantendo a estabilidade e liberando 100% dos recursos em seguida sem erros.**

- **Caso Inválido: Criei o diretório teste_invalido/ onde o cliente com limite 3 2 2 tentou requisitar 4 2 2. Confirmei que o programa barrou o pedido emitindo a mensagem exata: The customer 0 request 4 2 2 was denied because exceed its maximum allocation.**

8. Reflexão Final

A escolha do desenvolvimento incremental com testes modulares e a alocação dinâmica contígua de memória garantiram um código organizado e livre de vazamentos de memória (memory leaks), gerenciados via atexit(limpa_recursos) , gerenciados. Uma decisão técnica crucial foi o uso de mains temporários para validar isoladamente a leitura de arquivos e o algoritmo de segurança (esta_seguro) antes de fechar a integração final da main. O principal aprendizado durante o projeto foi a necessidade de precaver erros de escopo e ordenação

de funções no C11, além de assegurar o correto preenchimento da matriz NEED (Need = Max - Allocation) logo no início da execução. Em desenvolvimentos futuros, evitaria manter trechos temporários duplicados no mesmo arquivo durante os testes, adotando uma separação estrita de headers desde a primeira etapa para acelerar o processo de compilação e debugging.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Se dispusesse de mais duas horas de desenvolvimento, implementaria melhorias focadas em automação, modularização e robustez. Criaria um script em Bash (test_suite.sh) para automatizar a validação de múltiplos cenários de teste comparando o result.txt via diff, e refatoraria o arquivo monolítico banker.c em módulos separados (banker.h, parser.c, safety.c e main.c) para eliminar avisos e riscos de redefinição. Além disso, adicionaria manipuladores de sinais POSIX (SIGINT/SIGTERM) para garantir a limpeza de memória e remoção de arquivos incompletos em interrupções abruptas, finalizando com o cálculo dinâmico do alinhamento da tabela na função imprime_estado para suportar recursos com três ou mais dígitos.